

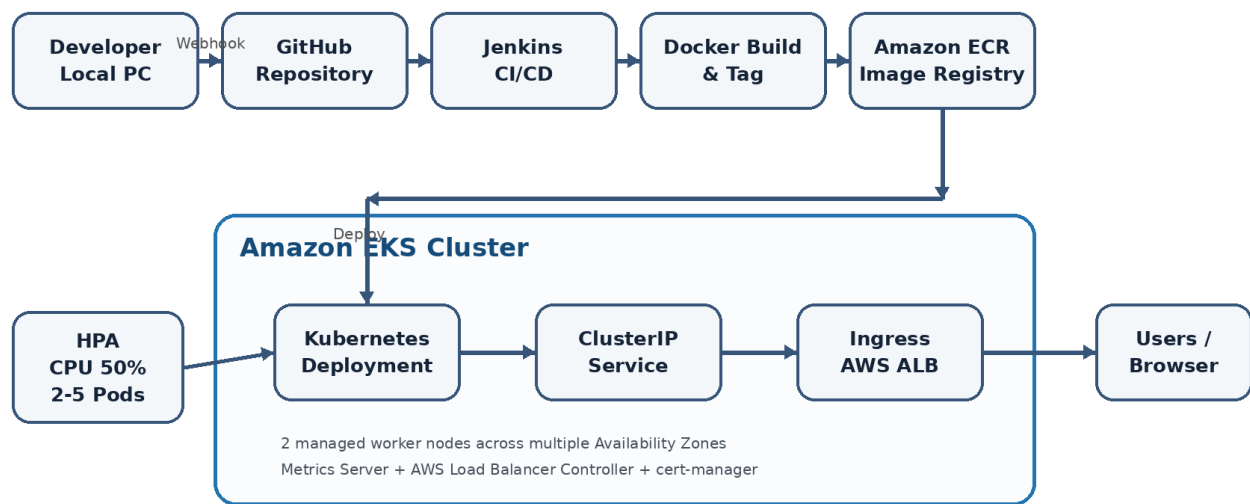
# SRE CI/CD & Kubernetes Auto Scaling Platform

*Production-Inspired Hands-On Project Documentation*

GitHub • Jenkins • Docker • Amazon ECR • Amazon EKS • ALB • HPA

## SRE CI/CD + Amazon EKS Auto Scaling Architecture

GitHub webhook -> Jenkins -> Docker -> ECR -> EKS -> ALB -> Browser | HPA 2-5 Pods



Final validation: HTTP 200 | Application Version 2.2.0 | Jenkins Build #12 successful

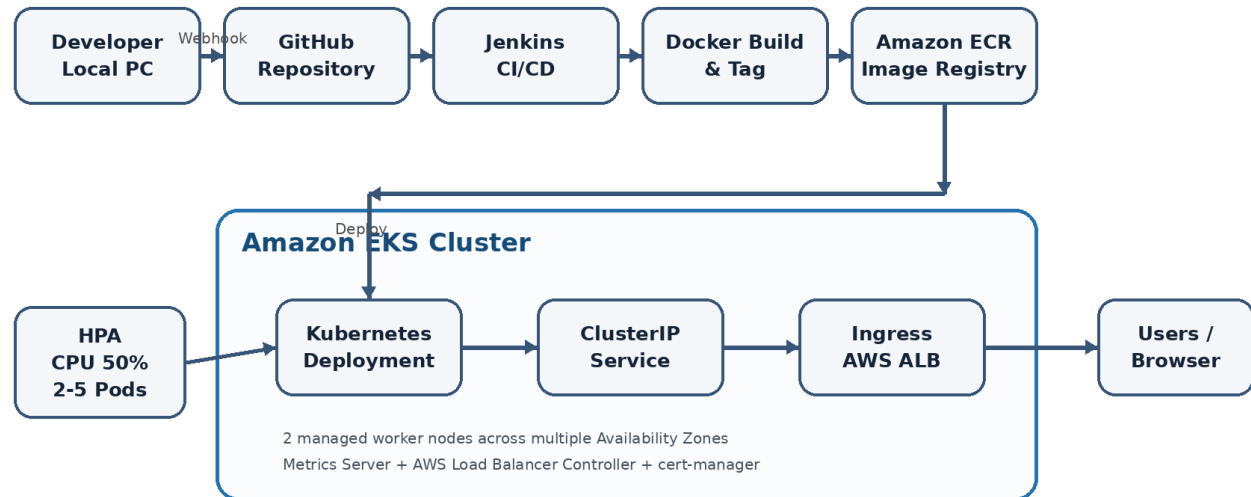
## 1. Project Objective

Build and validate an automated application delivery platform where a source-code change pushed to GitHub automatically triggers Jenkins, builds a Docker image, pushes it to Amazon ECR, deploys it to Amazon EKS, exposes it through an AWS Application Load Balancer, and supports Kubernetes Horizontal Pod Autoscaling.

## 2. Final Architecture

### SRE CI/CD + Amazon EKS Auto Scaling Architecture

GitHub webhook -> Jenkins -> Docker -> ECR -> EKS -> ALB -> Browser | HPA 2-5 Pods



Final validation: HTTP 200 | Application Version 2.2.0 | Jenkins Build #12 successful

## 3. Technology Stack

| Component                    | Purpose                                               |
|------------------------------|-------------------------------------------------------|
| GitHub                       | Source control and webhook trigger                    |
| Jenkins                      | Automated CI/CD orchestration                         |
| Docker                       | Container image build                                 |
| Amazon ECR                   | Private container image registry                      |
| Amazon EKS                   | Managed Kubernetes cluster                            |
| Kubernetes Deployment        | Runs and updates application Pods                     |
| ClusterIP Service            | Stable internal service endpoint                      |
| AWS Load Balancer Controller | Creates/manages ALB from Kubernetes Ingress           |
| Ingress / ALB                | Public HTTP entry point                               |
| Metrics Server               | Provides resource metrics to Kubernetes               |
| HPA                          | Scales application from 2 to 5 Pods at 50% CPU target |
| Nginx                        | Serves the static web application                     |

## 4. CI/CD Workflow

Developer change → Git push → GitHub webhook → Jenkins pipeline → Docker build → ECR login/tag/push → kubectl apply → current image rollout → Kubernetes rollout verification → ALB → browser.

Jenkins used the build number as the container image tag, providing a simple traceable version for each pipeline run.

## 5. Kubernetes Application Resources

- deployment.yaml - 2 application replicas with CPU/memory requests and limits.
- service.yaml - ClusterIP service on TCP/80.
- ingressclass.yaml - ALB IngressClass using ingress.k8s.aws/alb.
- ingress.yaml - Internet-facing ALB, IP target type, health check path '/'.
- hpa.yaml - minimum 2 replicas, maximum 5 replicas, 50% CPU utilization target.

## 6. Platform Configuration

- EKS cluster: sre-cicd-eks-cluster, Kubernetes 1.36, us-east-1.
- Managed node group: sre-eks-node-group; two t3.small worker nodes were used for final validation.
- Jenkins ran on Amazon Linux 2023 with Java, Git, AWS CLI, kubectl and Docker.
- AWS Load Balancer Controller used IAM/OIDC integration through a Kubernetes service account.
- Metrics Server enabled HPA metrics.
- EKS control-plane logging was enabled for API, audit, authenticator, controller manager and scheduler.

## 7. Key Troubleshooting Performed

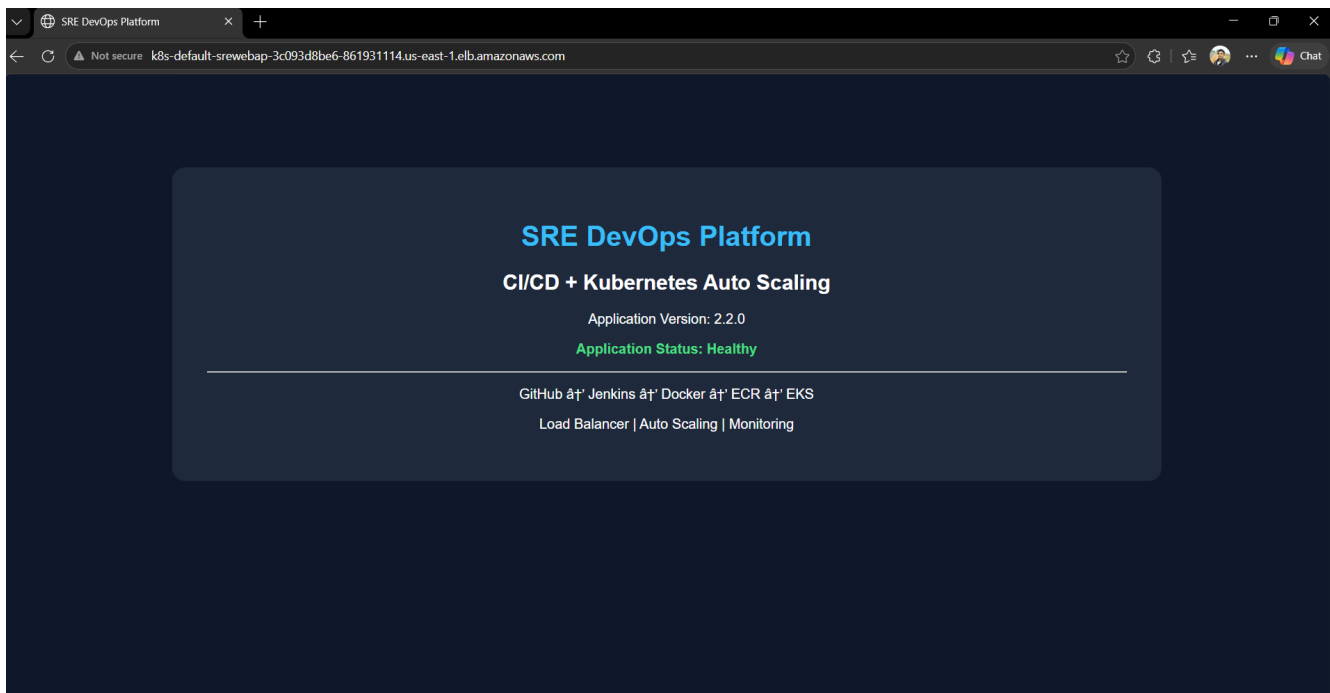
| Issue                    | Resolution                                                                                                                                           |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Worker node connectivity | Resolved EKS control-plane/node communication by correcting cluster security-group rules.                                                            |
| Pod capacity limitation  | Scaled the managed node group from one to two nodes so required system components could schedule.                                                    |
| ALB controller startup   | Controller could not discover the VPC through instance metadata; explicit AWS region and VPC ID arguments were added.                                |
| Ingress pipeline failure | Build failed because IngressClass 'alb' did not exist; added ingressclass.yaml and corrected manifest apply order.                                   |
| ALB DNS resolution       | Initial browser lookup returned NXDOMAIN; DNS cache was flushed and public DNS propagation was verified.                                             |
| Jenkins permissions      | Used scoped IAM/EKS access for ECR push and Kubernetes deployment; destructive EKS/ELB actions were intentionally unavailable from the Jenkins role. |

## 8. Final Validation Results

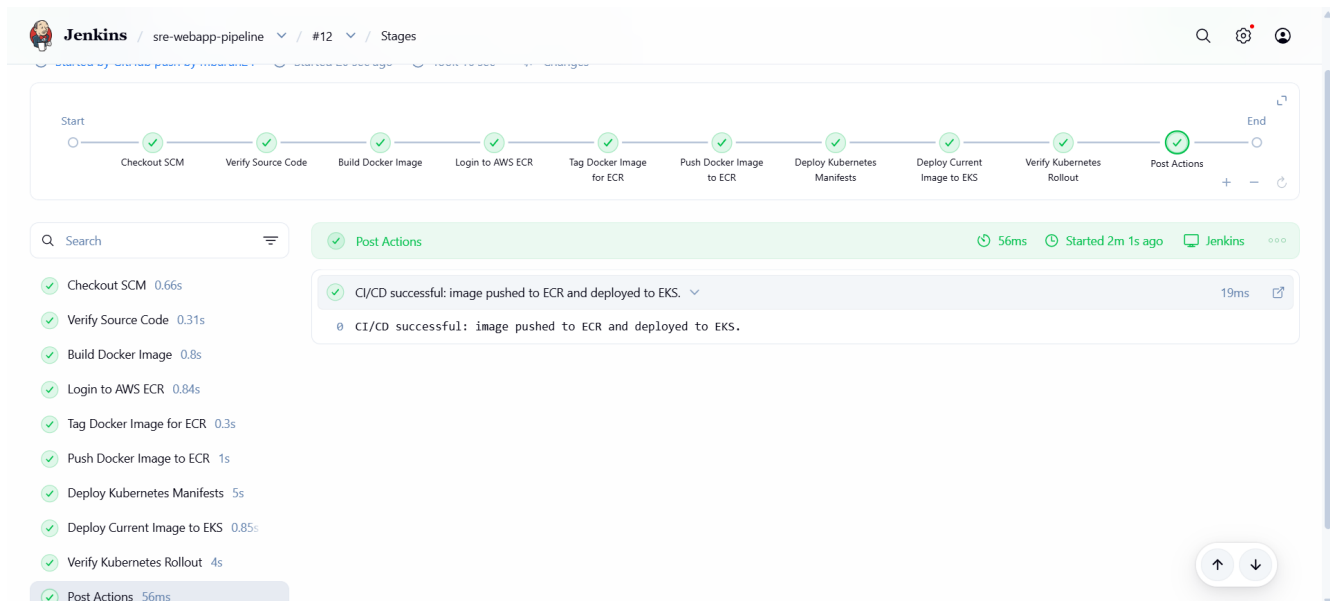
| Validation                | Result                                 |
|---------------------------|----------------------------------------|
| EKS worker nodes          | 2/2 Ready                              |
| Application Pods          | 2/2 Running                            |
| Deployment                | 2/2 Available                          |
| Service                   | ClusterIP on port 80                   |
| Ingress                   | Public ALB address assigned            |
| HPA                       | CPU 2% / 50%; min 2, max 5, replicas 2 |
| Public HTTP test          | HTTP 200                               |
| Measured response time    | 0.008554 seconds                       |
| Final application version | 2.2.0                                  |
| Final Jenkins pipeline    | Build #12 successful                   |

## 9. Deployment Evidence

Final browser validation - Application Version 2.2.0, Healthy



Jenkins Build #12 - all CI/CD stages completed successfully



## 10. Cleanup Performed

- Kubernetes application resources removed.
- Managed EKS node group deleted.
- EKS cluster deleted.
- Application Load Balancer confirmed removed.
- Amazon ECR repository deleted.
- Jenkins EC2 instance terminated.
- GitHub repository and local source files retained as portfolio/project evidence.
- IAM roles/policies may be retained without direct resource charges, but unused permissions should be reviewed periodically.

## 11. Project Outcome

### STATUS: SUCCESSFULLY IMPLEMENTED, VALIDATED AND CLEANED UP

The project demonstrated a complete automated delivery path from source control to a scalable Kubernetes workload. A final application change was delivered automatically through GitHub webhook and Jenkins, and Version 2.2.0 was successfully served through the AWS ALB. The final public endpoint returned HTTP 200.